Leakage Is a Moving Target: A Field Guide to Self-Deception in Differentiable Cross-Sectional Trading

Muthukumaran Navaneethakrishnan
Independent researcher

M. K. Haribalaji
Independent researcher

2026-06-06

Abstract We train neural models end-to-end against a profit-and-loss objective to forecast next-session cross-sectional positions on Indian large-caps (NSE; a 5-ticker universe, NIFTY 50, and NIFTY 100), across several signal families. No configuration produced a leakage-clean, post-cost edge above the bound consistent with chance. The contribution is not the null result but the methodology that earns it: a suite of negative-control gates run under strict walk-forward with bit-exact reproducibility, and an empirical taxonomy of the *escape routes* a differentiable optimiser takes when each prior leak is closed. We document three failure modes that survive naive evaluation: (i) the scale-invariance of a differentiable Sharpe objective, which lets the optimiser post a high Sharpe while driving absolute P&L to zero; (ii) cross-stock identity memorisation, in which per-entity structure is recovered as a label even from a time-permuted panel, invisible to a return-shuffle test but caught by static-feature and permutation controls; and (iii) the preprocessing-dependence of the leakage tests themselves — the power of a planted-cheat detector varies with the standardisation regime under test, so "no leak detected" is conditional on a choice the experimenter rarely reports. We give bootstrap-calibrated null bounds for small holdout regimes, where the closed-form Sharpe standard error under-states the true null by roughly six-fold. We position the work against the concurrent structured-null falsification of Nikolopoulos and the selection-bias line of Bailey and López de Prado, and release every gate as runnable code with synthetic fixtures that reproduce each claim.

Reproducibility. Every gate named below is a runnable file in `../src/leakage_harness/`; every illustrative number is reproduced by a script in `../demos/` against seeded synthetic panels — no market data, no credentials, no network. See `../README.md` for the finding → file → demo map.

1. Introduction — the fake-Sharpe problem

A backtest is a measurement instrument pointed at a noisy, non-stationary, adversarially-sampled process, and almost every degree of freedom in its construction biases the reading upward. The literature on this is mature: Bailey and López de Prado formalised how testing many strategy configurations inflates the best observed Sharpe (the Deflated Sharpe Ratio, and the Probability of Backtest Overfitting) (Bailey and López de Prado 2014; Bailey et al. 2016); López de Prado

catalogued the temporal-leakage failures of naive cross-validation and proposed purged and combinatorial schemes (López de Prado 2018); Kaufman et al. gave the canonical taxonomy of data leakage in predictive modelling (Kaufman et al. 2012).

End-to-end *differentiable* trading raises the stakes. Rather than fitting a return forecast and then sizing positions, a single network is optimised directly against a P&L or risk-adjusted-return objective computed by a differentiable backtest (Zhang et al. 2020). This is expressive and convenient, and it is also a remarkably efficient search for whatever artifact in the data most cheaply reduces the loss. When the objective is a Sharpe ratio and the panel is cross-sectional, the cheapest artifacts are subtle: they are not "the model peeked at tomorrow's price" but "the loss is invariant to a degree of freedom we forgot to constrain" or "per-entity bookkeeping has quietly become a label."

This paper is a field guide to those artifacts, written from a research program that set out to find a tradeable next-session edge on Indian large-caps and instead found a null — repeatedly, across momentum, news, per-stock, linear, sector-demeaned, and post-earnings-drift signal shapes, at universes of 5, 50, and 100 tickers. The null itself is unremarkable and we do not over-claim it; markets being hard to beat is the prior. What is worth recording is *how* each apparent edge turned out to be self-deception, and the specific negative-control gate that exposed it. Leakage, in this setting, is a moving target: every gate we added closed one escape route and revealed the next.

1.1 What is, and is not, novel here

We are explicit about prior art because the honest contribution is narrow. Concurrent with this work, Nikolopoulos developed a falsification audit that tests complete predictive workflows against synthetic reference classes — including zero-predictability environments and microstructure placebos — and quantifies selection-induced inflation (Nikolopoulos 2026). That work covers the *general* frame we also arrived at: structured nulls plus a selection-aware ceiling. We do not claim the frame.

What we believe is not already published in this combination is the *differentiable, cross-sectional-panel-specific* cluster:

- the scale-invariance pathology of a differentiable Sharpe objective and the magnitude co-condition that detects it (§3);
- cross-stock identity memorisation as a panel batch-effect that a return-shuffle test cannot see, caught by a static-feature negative control and a ticker-permutation control (§4);
- the preprocessing-dependence of leakage-test power — a reflexive caveat on the detectors themselves (§5);
- bootstrap-calibrated null bounds for the small-holdout regime (§6).

Each of these has neighbours we cite rather than claim: the scale-invariance point is adjacent to the general scale-sensitivity of Sharpe; the identity channel is the *batch-effect* problem (Leek et al. 2010) instantiated in equities and adjacent to instance-normalisation critiques (Kim et al. 2022); the preprocessing caveat is a sharper statement of "preprocessing affects detectability." The synthesis, and the runnable gates, are the deliverable.

2. The harness

The substrate is a walk-forward train-and-backtest loop (`walk_forward.py`). For each holdout day `t`, a model is retrained on the trailing window `features[t-n_train:t]`, positions are predicted from `features[t]`, and per-step P&L is realised as `positions[t] · returns[t+1]`. The position emitter `StrategyNet` (`strategy_net.py`) is market-neutral by construction — its final operation subtracts the cross-sectional mean, so the model can only express *relative* views:

```
def forward(self, x):                  # x: [T, S, F] -> [T, S]
    z = self.net(x).squeeze(-1)
    return z - z.mean(dim=-1, keepdim=True)
```

This is not a regularizer to be tuned away; it is a hard contract, and it matters for the leakage story because it forces every artifact the optimiser finds to be a *cross-sectional* one.

A gate is any function returning a uniform verdict, `LeakageResult(name, passed, value, expected, notes)`. Gates come in two moral categories. **Positive controls** plant a known leak and must fire — if they do not, the harness is blind. **Negative controls** destroy a specific kind of signal and must *not* fire — if they do, the model was relying on a channel it should not have. The harness is only trustworthy when its positive controls fire and its negative controls stay silent on the same model and data.

One implementation detail recurs below. Losses historically took only the P&L vector; position-aware losses additionally need the positions. Rather than thread an argument everywhere, the trainer inspects the loss signature and dispatches accordingly (`walk_forward.py`):

```
sig = inspect.signature(loss_fn)
if "pos" in sig.parameters:
    return loss_fn(pnl, pos=pos)
return loss_fn(pnl)
```

This tiny mechanism is what lets the position-floor penalty of §3 coexist with pure-P&L losses without a breaking change to the call sites.

3. Escape route I — scale-invariance collapse

The annualised Sharpe ratio is invariant to a positive rescaling of P&L: multiply every position by a constant and the ratio is unchanged. A loss of `-Sharpe` therefore gives the optimiser no reason to take *any* particular position size. In practice the optimiser exploits this: it shrinks positions toward zero while keeping their *direction* aligned just enough to hold the ratio up. The result is a strategy with a respectable reported Sharpe and a mean P&L numerically indistinguishable from zero — an "edge" that trades nothing.

Sharpe alone cannot detect this, precisely because it is the invariant being gamed. The fix is a magnitude co-condition. In the look-ahead positive control (`leakage_tests.py`), passing requires not only that the planted cheat lift the Sharpe past a threshold but that the cheat run's absolute mean P&L dominate the honest run's by an order of magnitude:

```
ratio = abs(r.mean_pnl) / max(abs(mean_pnl_honest), 1e-8)
passed = (r.sharpe >= 5.0) and (ratio >= 10.0)
```

and, on the training side, a position-floor penalty that sees magnitude directly (`losses.py`):

```
def sharpe_with_position_floor(pnl, pos, floor=0.05, alpha=0.5, ...):
    sharpe = pnl.mean() / (pnl.std(unbiased=False) + eps) * sqrt(ppy)
    penalty = alpha * relu(floor - pos.abs().mean())
    return -sharpe + penalty
```

The lesson generalises beyond this loss: *position regularisation belongs in the harness contract, not inside any single objective.* A risk-adjusted objective is a ratio, and ratios invite the optimiser to collapse the denominator-free degree of freedom you left open.

4. Escape route II — cross-stock identity memorisation

The most instructive failure was a model that scored a large Sharpe on a panel whose returns had been **permuted along the time axis** — a setting in which, by construction, no time-aligned signal can survive. The model was not using the future. It had memorised *which ticker is which* and bet on persistent per-entity structure: long-run volatility, baseline news density (orders of magnitude apart across names), fundamental levels. Permuting time leaves that cross-sectional structure intact, so a return-shuffle test — the standard first-line leakage check — passes the model through.

This is the batch-effect problem (Leek et al. 2010) in equity clothing: a per-group bookkeeping quantity becomes a label the model can exploit — a shortcut in the sense of Geirhos et al. (2020), a decision rule that scores well in-sample yet rests on a spurious cue rather than the intended signal. We found it acutely when features were standardised *per stock*: the per-ticker scale `σ_ticker` encodes ticker identity, and a model can recover the cross-sectional ranking from a noise-only panel. Swapping per-stock standardisation for a global causal standardisation dropped the static-feature Sharpe magnitude on that channel from roughly 2.6 to 0.9 — a leak hiding inside a preprocessing choice that looks like hygiene.

Two negative controls catch what the shuffle misses. The **static-features** gate (`stability_tests.py`) replaces every feature with its per-stock time-mean, destroying all time variation while preserving identity; a real time-edge goes flat, an identity-memoriser keeps its Sharpe:

```
static_feat = features.mean(dim=0, keepdim=True).expand_as(features)
r = walk_forward(static_feat, returns, ...)
passed = abs(r.sharpe) < max(1.0, 2.0 * sharpe_se)
```

The **permutation-invariance** gate retrains on relabelled tickers: the physics from features to returns does not care about names, so a genuine edge survives renaming and a memoriser collapses. On synthetic panels the contrast is stark (`demos/02_identity_leakage.py`): a pure time-edge panel yields a static-feature Sharpe near zero (gate passes), while a panel with a persistent per-stock drift exposed through a constant identity feature yields a static-feature Sharpe of roughly +30 (gate fails, correctly reporting the leak).

A partial mitigation we adopted during training is ticker dropout: masking a random fraction of tickers each step and re-imposing the zero-sum constraint, which strips some of the identity

channel. It reduces but does not eliminate the leak — the model still extracts identity from feature covariance — which is why the controls, not the mitigation, are the load-bearing part.

5. Escape route III — regime-dependent detector blindness

The uncomfortable finding is reflexive: the *power* of a leakage test depends on the preprocessing regime it is run under. Consider the two regimes we used. With **raw** features, the return-shuffle negative control is sensitive and tends to fire on structural leaks — but a planted look-ahead cheat, measured at raw return scale, can be numerically swamped and the positive control under-fires. With **standardised** features, the cheat column is compressed toward the global noise scale after normalisation, so the planted-cheat positive control loses power and reports "no leak" — while the shuffle control behaves differently again.

We address the most mechanical version of this in the look-ahead control by passing the same standardisation transform to the augmented tensor, so the planted cheat lands on the same footing as the honest features (`leakage_tests.py`, the `transform_fn` argument). But the general point stands and is not fully solved by any single fix: **no single configuration made every gate maximally powerful at once.** A `passed = False` from a planted-cheat detector is therefore not unconditional evidence of a clean pipeline; it is evidence conditional on a preprocessing choice that is rarely reported alongside the result. We recommend reporting leakage-test verdicts *with* the regime under which they were obtained, and running positive controls under each regime a paper actually uses.

6. Calibrating the null

A leakage gate compares an observed Sharpe to a bound on the null. The convenient closed form is the standard error of an annualised Sharpe, `sqrt(252 / N)` for `N` effective samples. In the small-holdout, small-universe regime typical of careful walk-forward on a year or two of data, this badly under-states the true spread of the null. Empirically, the standard deviation of the Sharpe across return-shuffle seeds on pure noise was about six times the closed-form SE, and only a minority of shuffle seeds fell inside the theoretical `2·SE` band.

The remedy is to calibrate the bound by bootstrap rather than formula. The return-shuffle gate optionally runs `K` shuffles, takes the empirical mean and standard deviation of the resulting Sharpes, and uses `2·max(empirical_std, SE)` as the bound, floored at the closed form so it is never looser than necessary (`leakage_tests.py`). The reproducibility control still holds because each shuffle seed is fixed. On a genuine synthetic time-edge (`demos/01_structural_vs_time_leakage.py`), the honest walk-forward Sharpe is solidly positive while the shuffled Sharpe sits inside the bootstrap-calibrated null band — the gate correctly reports no structural leakage, where the naive `2·SE` bound would have raised a false alarm.

A related hygiene point: with cross-seed Sharpe standard deviation often above 1.5, a five-seed average is too thin to trust a borderline-positive verdict. One headline result that read `+0.31` at five seeds regressed to `-0.07` at twenty. We default to twenty seeds before treating any small-positive result as evidence.

7. Results — the honest null

Under the full gate suite — return-shuffle and look-ahead and future-news controls, static-feature and permutation-invariance and window-robustness controls, bit-exact reproducibility, bootstrap-calibrated bounds, and twenty-seed averaging — no signal family produced a leakage-clean post-cost Sharpe distinguishable from chance, at any of the three universes. Apparent edges, when they appeared, were traced to one of the escape routes above and vanished once the corresponding control was enforced.

We note one signed empirical observation that survived auditing, reported as an observation rather than an edge: daily news tone on Indian large-caps (from GDELT) was weakly *contrarian*, not momentum-like — the correlation between same-day tone and next-day return was slightly negative (approximately −0.016 overall, strengthening to about −0.11 on large-tone days). The plausible mechanism is that news indexes a move after it has happened, and the next day partially mean-reverts. It is too weak, post-cost, to trade.

8. Limitations and threats to validity

The runnable gates in this repository operate on synthetic planted-signal fixtures, chosen so that every claim reproduces from a clean clone; the empirical null was obtained on real NSE data in the originating research program and is summarised, not re-derived, here. The escape-route taxonomy is empirical and almost certainly incomplete — it lists the routes *this* optimiser took on *this* data, and a different architecture or objective will find others; that is the point of the title. The novelty claims of §1.1 rest on a literature search, not an exhaustive systematic review, and the most defensible reading is "not found in this combination" rather than "first." Finally, the gates establish the *absence* of specific artifacts, not the presence of an edge; passing every control is necessary, never sufficient.

9. Conclusion

The methodology generalises; the data edge did not. For end-to-end differentiable cross-sectional models, the loss is an adversary as much as an objective: it will collapse any unconstrained scale, exploit any per-entity bookkeeping, and hide inside any preprocessing choice that doubles as a leak. The practical upshot is a discipline — positive controls that must fire, negative controls that must stay silent, bounds calibrated to the regime, and verdicts reported together with the preprocessing they were measured under. We release the gates so the next person searching this space can fail honestly, and faster.

Data and code availability

All gates and demos are available at https://github.com/muthuishere/leakage-moving-target-research and archived at Zenodo (DOI: TODO). The demos reproduce every illustrative number against seeded synthetic fixtures with no external data dependency.

References

Bailey, David H., Jonathan Borwein, Marcos López de Prado, and Qiji Jim Zhu. 2016. "The Probability of Backtest Overfitting." *Journal of Computational Finance* 20 (4): 39–69.

Bailey, David H., and Marcos López de Prado. 2014. "The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting, and Non-Normality." *The Journal of Portfolio Management* 40 (5): 94–107.

Geirhos, Robert, Jörn-Henrik Jacobsen, Claudio Michaelis, et al. 2020. "Shortcut Learning in Deep Neural Networks." *Nature Machine Intelligence* 2 (11): 665–73. https://doi.org/10.1038/s42256-020-00257-z.

Kaufman, Shachar, Saharon Rosset, Claudia Perlich, and Ori Stitelman. 2012. "Leakage in Data Mining: Formulation, Detection, and Avoidance." *ACM Transactions on Knowledge Discovery from Data* 6 (4): 1–21.

Kim, Taesung, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. 2022. "Reversible Instance Normalization for Accurate Time-Series Forecasting Against Distribution Shift." *International Conference on Learning Representations (ICLR)*.

Leek, Jeffrey T., Robert B. Scharpf, Héctor Corrada Bravo, et al. 2010. "Tackling the Widespread and Critical Impact of Batch Effects in High-Throughput Data." *Nature Reviews Genetics* 11 (10): 733–39. https://doi.org/10.1038/nrg2825.

López de Prado, Marcos. 2018. *Advances in Financial Machine Learning*. John Wiley & Sons.

Nikolopoulos, Sotirios D. 2026. "Spurious Predictability in Financial Machine Learning." *arXiv Preprint arXiv:2604.15531*.

Zhang, Zihao, Stefan Zohren, and Stephen Roberts. 2020. "Deep Learning for Portfolio Optimization." *The Journal of Financial Data Science* 2 (4): 8–20.